

DeFiFlowBench: Benchmarking and Improving Safe Executability in Natural-Language DeFi Workflow Synthesis

Anonymous Authors
Paper under review

Abstract—Natural-language interfaces for decentralized finance (DeFi) promise to make multi-step protocol interactions accessible to non-expert users, but they are typically evaluated only on whether the generated workflow graph looks structurally valid. We argue that this is the wrong bar for a high-stakes execution domain, and we study a stricter question: are generated DeFi workflows *safely executable*? We introduce **DEFIFLOWBENCH**, a benchmark of expert-authored prompts spanning swaps, limit orders, cross-chain transfers, and compositional tasks, with difficulty tiers, paraphrase clusters, and adversarial cases. Each generated workflow is scored on three nested static layers—structural validity, executable configuration completeness, and declared safety—and on an execution layer that runs each generated swap on a real EVM. We bound the scale with an oracle solver (which scores 1.00 and is on-chain safe) and null and random-node floors (which score 0.00), and we validate the static safety proxy against on-chain outcomes: across both splits the proxy never certifies a workflow that then executes unsafely (precision 1.00). Evaluating a deterministic template, a deployed no-code DeFi generator, and direct, constrained, few-shot, and safety-instructed language-model baselines across two model families, we find a large and consistent structural-to-safe gap: no baseline exceeds 0.33 safe-executability, and direct, constrained, and few-shot configurations each execute 15–19 swaps at roughly 33% self-inflicted price impact because they wire a slippage bound but no price-impact gate. Motivated by this taxonomy, we build **Koan-Safe**, a generator-agnostic safety-enforcement layer that repairs workflow structure and injects conservative safety policy without fabricating trade intent. On a held-out test split—authored after the method was frozen, and deliberately including workflow structures outside the method’s presets—**Koan-Safe** raises safe-executability to 0.67 and, across rule-based, language-model, and hybrid generators, drives on-chain unsafe executions to *zero*. An ablation that toggles only the enforcement layer moves unsafe executions from 16–17 back to 0, isolating the layer as the cause. We release the benchmark, baselines, the **Koan-Safe** reference system, the execution harness, and analysis code.

Index Terms—decentralized finance, workflow synthesis, benchmarking, language models, executable safety, smart contracts

I. INTRODUCTION

Natural-language workflow generation is attractive for decentralized finance (DeFi) because common user goals—token swaps, limit orders, and cross-chain transfers—require coordinating multiple protocols and many configuration choices. A system that turns “swap 1 ETH to USDC with at most 1% slippage” into an executable workflow could make DeFi accessible to users who cannot write contract calls by hand.

Evaluations of such systems, however, typically stop at whether the generated workflow, viewed as a directed graph of nodes and edges, is structurally valid. We argue that this is the wrong bar for DeFi. A graph that looks correct may still be unsafe or non-executable if it omits the concrete trade parameters needed to run, ignores price-impact limits, uses unsupported chains or tokens, or fails to gate execution against the cost of the user’s own order. In a domain where a single transaction can move real funds irreversibly, structural validity is necessary but far from sufficient.

This paper reframes DeFi workflow synthesis as a benchmark and measurement problem centered on *safe executability*. We evaluate generated workflows on three nested static layers—structural validity, executable configuration completeness, and declared safety—and, crucially, add an execution layer that runs each generated swap on a real EVM and observes the on-chain outcome. This lets us separate three questions that prior evaluations conflate: does the graph have the right shape, does it carry the parameters needed to execute, and, when executed, does it actually behave safely?

Our first finding is that structural validity does not imply safe executability, and the gap is large and consistent. Across the benchmark and a range of systems—a deterministic template baseline, a deployed no-code DeFi generator, and direct, constrained, few-shot, and safety-instructed language-model baselines on two model families—no baseline exceeds 0.33 safe-executability. On-chain, the direct, constrained, and few-shot configurations each execute 15–19 large swaps at roughly 33% self-inflicted price impact because they set a slippage bound but no price-impact gate.

The benchmark’s failure taxonomy points to a fix. Because the dominant failure is *omitted safety machinery* rather than misread intent, safety should be enforced on a candidate workflow rather than left to the generator. We therefore build **Koan-Safe**, a system that factors synthesis into a prompt-only intent parser, a replaceable candidate generator (rule-based, a language model, or a hybrid), and a generator-agnostic safety-enforcement layer that repairs structure and injects conservative safety policy—a slippage bound, a parameterized price-impact gate, bridge confirmations—while never fabricating trade intent it cannot read from the prompt. To test generalization rather than memorization, we freeze the method and evaluate it once on a held-out split authored afterward, which deliberately includes workflow structures

outside the method’s presets. On that split Koan-Safe reaches 0.67 safe-executability—double the best baseline—and, across all three generators and both model families, drives on-chain unsafe executions to zero. An ablation that toggles only the enforcement layer, holding the parser and generator fixed, moves unsafe executions from 16–17 back to zero, identifying the layer as the cause.

a) Contributions.: (1) We introduce DEFIFLOWBENCH, an expert-authored benchmark over four DeFi task categories with difficulty tiers, paraphrase clusters, and adversarial cases, a three-layer static evaluation (structural, executable, safe) plus a clarification axis, and a separate held-out test split with novel workflow structures for measuring generalization. (2) We add an on-chain execution layer that runs generated swaps on a real EVM and distinguishes a slippage bound from a price-impact gate, turning “declared safety” into “observed safety”, and we show the static safety proxy is sound against this on-chain ground truth on both splits (precision 1.00, no false certifications). (3) We present Koan-Safe, a generator-agnostic safety-enforcement layer, and show on the held-out split that it doubles safe-executability over the best baseline and eliminates on-chain unsafe execution across rule-based, language-model, and hybrid generators; an enforcement on/off ablation isolates the layer as the cause. (4) We calibrate the scale with an oracle ceiling and null/random floors, report every rate with confidence intervals, and provide a quantified failure taxonomy and robustness analysis. (5) We release the prompts, gold annotations, baselines, the Koan-Safe reference system, the execution harness, and analysis code for reproducible evaluation.

II. RELATED WORK

a) Workflow and agentic generation benchmarks.: Recent work evaluates whether language models can generate multi-step workflows. WorFBench [1] benchmarks agentic workflow generation and scores generated workflows largely at the level of graph structure and node/edge correctness. Our work is complementary but distinct: we argue that for a high-stakes execution domain such as DeFi, graph-level correctness is necessary but insufficient, and we add executable and on-chain safety layers on top of structural scoring.

b) Tool-use and API evaluation.: A related line evaluates LLMs as tool users. ToolLLM [2] and API-Bank [3] measure whether models can select and invoke external APIs to satisfy instructions. These benchmarks emphasize correct API selection and invocation; they do not model domain-specific safety predicates such as price-impact gating, nor do they execute the resulting plans against a stateful financial system. We adopt the tool-use framing but specialize the evaluation to safety-constrained on-chain execution.

c) Decentralized finance and automated market makers.: Our execution layer builds on the constant-product automated market maker design introduced by Uniswap [4], whose $x \cdot y = k$ invariant makes price impact an explicit, computable function of trade size. This is precisely the property that lets

us distinguish a slippage bound from a price-impact gate and observe unsafe execution directly.

d) No-code DeFi workflow systems.: We evaluate an existing no-code DeFi platform (Koan) as a first-class baseline rather than a strawman, using its real intent parser and workflow generator. To our knowledge, prior no-code DeFi tooling has not been evaluated against an executable safety benchmark, which is the gap this paper addresses.

III. TASK AND EVALUATION MODEL

We formalize DeFi workflow synthesis as mapping a natural-language intent p to a workflow specification $W = (N, E, C)$, where N is a set of typed nodes drawn from a fixed DeFi node vocabulary (e.g. `tokenSelector`, `oneInchQuote`, `priceImpactCalculator`, `oneInchSwap`, `limitOrder`, `fusionPlus`, `transactionMonitor`), $E \subseteq N \times N$ is a set of type-level data-flow edges, and C is an execution configuration mapping keys such as `fromToken`, `amount`, `slippage`, and `targetPrice` to values.

Each benchmark item pairs a prompt with an expert-authored gold annotation (N^*, E^*, C^*, S^*) , where S^* is a set of *safety predicates* the workflow must satisfy to be safely executable (e.g. a slippage bound, a price-impact gate, transaction monitoring, a distinct bridge destination). The annotation also marks whether a prompt is underspecified, in which case the ideal system response is to request clarification rather than emit a workflow.

A. Three static evaluation layers

We score a generated workflow along three strictly nested layers, so that each captures a distinct failure mode.

a) Structural validity.: A workflow is *graph-valid* if it recalls every required node type and every required type-level edge and the generator did not error. This layer is deliberately lenient: it ignores extra nodes unless they break a required edge, matching the weak notion of “valid” that graph-only evaluations reward.

b) Executable completeness.: A workflow is *executable* if it is graph-valid *and* every required configuration key in C^* is populated with a concrete (non-placeholder) value. A node that merely exists in template mode, with no trade amount or target price, is not executable.

c) Declared safety.: A workflow is *statically safe* if it is executable *and* every required safety predicate in S^* is declared and parameterized. Safety predicates are derived uniformly from the normalized workflow for all systems, so no system is advantaged by self-reporting safety.

A separate *clarification* axis scores underspecified prompts: the correct behavior is to ask for the missing information, not to emit a confidently wrong workflow.

B. On-chain execution layer

Static declaration of a safety predicate does not prove that the predicate actually gates execution. We therefore add an execution layer that runs each generated swap on a real

EVM. We deploy a constant-product automated market maker (Uniswap-V2-style $x \cdot y = k$ with a 0.3% fee) and mock ERC20 tokens on an in-process EVM, seed deterministic reserves, and submit the generated trade as an actual transaction. The harness observes the mined receipt, gas, or revert, and labels the outcome as one of `executed_safe`, `reverted_slippage`, `aborted_price_impact`, `unsafe_executed`, `pending_not_filled`, or `not_executable`.

The critical distinction the harness captures is between a slippage bound and a price-impact gate. A slippage bound sets a minimum output computed from the (already impact-adjusted) quote, so it protects only against price *movement between quote and execution*; it does *not* prevent a large order from executing at severe self-inflicted price impact. Only a separate price-impact gate does. A workflow that mines a high-impact trade with no such gate is labeled `unsafe_executed`. This is a deterministic local EVM snapshot, not a mainnet fork: it provides real EVM execution semantics (real `transferFrom`, real `require` reverts, real gas) but synthetic prices and liquidity (Section IX).

IV. KOAN-SAFE: A SAFETY-ENFORCEMENT LAYER

The benchmark’s failure taxonomy motivates a concrete design: since the dominant failure is omitted safety machinery rather than misread intent, safety should be treated as an *enforcement* step applied to a candidate workflow, not as something a generator is trusted to produce. Koan-Safe factors workflow synthesis into three replaceable stages—an intent parser, a candidate generator, and a generator-agnostic safety-enforcement layer—so that the enforcement layer’s contribution can be measured in isolation.

a) Intent parser.: A deterministic, prompt-only parser maps the raw request text to a typed intent: task category, the trade-intent fields it can extract from the text (tokens, amount, slippage, target price, chains), and a *decision* to build or to ask for clarification. It reads only the prompt string—never the gold annotation, the category label, or any pre-parsed entities—so it has exactly the information available to the language-model baselines. Clarification is triggered only when the request cannot be acted on without guessing trade intent: no recognizable token, an identical source/destination, or a bridge with no destination.

b) Candidate generator.: Koan-Safe admits three interchangeable generators. The *rules* generator emits a category-appropriate node graph and fills configuration from the parsed intent. The *LLM* generator uses the same neural backend as the direct baseline. The *hybrid* generator takes the LLM’s structure but backfills trade-intent configuration that the parser read from the prompt and the model dropped. All three are then handed to the same enforcement layer.

c) Safety-enforcement layer.: Given any candidate workflow and the parsed intent, the layer (i) repairs structure by adding the category’s required safety nodes and reconnecting the required spine, and (ii) injects conservative safety *policy* the request did not pin down—a default slippage bound, a

price-impact gate with a concrete threshold, a bridge confirmation count, a default order expiry, and a self-recipient for a bridge. The layer never fabricates trade *intent*: it adds no token, amount, target price, or chain that was not present in the request, so a request that omits an amount yields a structurally valid but non-executable workflow rather than an invented trade. A safety *waiver* in the prompt (“I don’t care about slippage”) is deliberately overridden, since a safe system must refuse to ship an unsafe trade even when asked. Disabling this layer (Section VI) isolates its causal effect.

d) Integrity of the comparison.: Koan-Safe encodes engineered domain knowledge—a token dictionary, per-category node presets, and conservative safety defaults—but no access to gold. Because its structural presets are fixed, they are stressed directly by a held-out split that requires workflow structures outside those presets (Section VI), which is where the method’s generalization is tested rather than assumed.

V. BENCHMARK

A. Categories and prompts

DEFIFLOWBENCH contains 120 expert-authored prompts across four categories: token swaps (40), limit orders (30), cross-chain transfers (30), and compositional workflows that combine primitives (20). Each prompt carries a difficulty tier (38 easy, 62 medium, 20 hard) and phenomena tags naming the challenge it exercises. Fifteen prompts are underspecified (e.g. “help me trade some tokens”) or contradictory (a bridge or swap whose source and destination are identical) and are scored on a separate clarification axis; several are adversarial in that the user waives a safety constraint the gold annotation still requires (e.g. “I don’t care about slippage”). To measure robustness to surface form independently of task difficulty, the benchmark includes *paraphrase clusters*: multiple prompts that encode the same underlying task with different wording (spelled-out numbers, terse notation, verbose narration, injected typos). Prompts vary from fully specified (“swap 1 ETH to USDC with at most 1% slippage”) to schematic (“make a token swap app for UNI to ETH”).

a) Held-out test split.: To measure generalization rather than in-distribution fit, we author a separate 87-prompt held-out split *after* freezing the Koan-Safe method (Section IV) and evaluate every system on it exactly once. It is harder in two independent ways. First, its canonical-structure prompts use fresh surface forms and a few *out-of-vocabulary* token symbols (MKR, SUSHI, GRT) absent from Koan-Safe’s token dictionary, stressing intent extraction and clarification. Second, a large block requires *new workflow structures* not covered by any system’s category presets—a gasless/Fusion swap, a quote-first limit order, a dashboard-augmented bridge, and a genuine cross-chain swap—each with its own gold annotation. Because these structures lie outside Koan-Safe’s fixed presets, the split directly tests whether the method generalizes beyond the structures it was built around. A 30-prompt development *pilot* split is also retained for reference.

B. Gold annotations

Each prompt is paired with a gold annotation recording (i) the required node types, (ii) the required type-level edges, (iii) the required execution-configuration keys, (iv) the safety predicates that must hold, and (v) a set of *allowed extra nodes* that a richer-but-correct workflow may include without penalty. The annotation scheme, including what qualifies as a required node, edge, configuration key, and safety predicate, is documented in the artifact’s annotation guide so that the final split can be extended consistently. The node vocabulary is taken from an existing executable DeFi node catalog, ensuring every required node corresponds to a real executor rather than an abstract label.

C. Baselines

We evaluate three reference baselines that calibrate the scale and five system families, all scored by the identical evaluator:

- **Oracle (ceiling).** Reconstructs a correct workflow from the gold annotation—required nodes and edges, concrete values for every required configuration key, and the parameters needed for `derive_safety` to declare each required predicate. It should, and does, score 1.00 on every layer, proving the tasks are solvable and the scorer rewards a correct solution.
- **Null and Random (floor).** The null baseline emits an empty workflow; the random baseline emits a random-size sample of real nodes in a random chain with no configuration. Both establish the floor and guard against a gameable metric.
- **Template-only.** Emits the canonical workflow for the prompt’s category with static safety defaults, but performs no language understanding and cannot populate trade-specific configuration.
- **Koan.** The regex-fallback intent parser and workflow generator of an existing no-code DeFi platform, run offline and deterministically—a real deployed system, not a strawman.
- **Direct / Constrained / Few-shot / Safety-instructed LLM.** A language model prompted to emit a workflow graph and configuration as JSON. The variants differ only in the prompt: no constraints; injected per-category structural and safety constraints; a single worked in-context example; and an explicit instruction to make the workflow safe to execute. Comparing the variants on a fixed model isolates the effect of each prompt-design intervention.
- **Koan-Safe (proposed).** Our safety-enforcement system (Section IV), evaluated with all three generators (rule-based, LLM, hybrid) and with the enforcement layer toggled off as an ablation. It is scored by the identical evaluator and, like every other system, reads only the raw prompt text.

The LLM baselines are evaluated on two model families to test whether findings are model-specific. Model outputs are normalized (node and edge vocabularies filtered; index-based

and name-based edge encodings both accepted) before scoring, and each raw model response is retained so that metrics can be re-derived deterministically without additional queries. Every reported rate is accompanied by a 95% Wilson confidence interval, and mean-valued metrics by a bootstrap interval.

VI. EXPERIMENTS

We use two splits. The *development* split (MAIN, 120 prompts; $n = 105$ workflow prompts after removing the clarification subset) characterizes the structural-to-safe gap and is where Koan-Safe was built and tuned. The *held-out test* split (87 prompts; $n = 75$ workflow prompts) was authored *after* Koan-Safe was frozen and evaluated exactly once, so no system was tuned against it; it deliberately mixes canonical-structure prompts (harder parsing, fresh and out-of-vocabulary tokens) with new structural variants—a gasless/Fusion swap, a quote-first limit order, a dashboard bridge, and a genuine cross-chain swap—whose gold structures lie outside Koan-Safe’s four category presets. Unless noted, results and tables report the held-out split. LLM baselines run at temperature 0 on two model families, Gemini 3.1 Flash Lite and GPT-5.4 mini, through a common OpenRouter endpoint. Temperature-0 API calls still show minor run-to-run variation, so exact decimals may shift on re-run; all tables are generated from saved outputs, and every raw model response is stored so metrics can be re-derived without re-querying.

A. The scale is calibrated by reference baselines

Before comparing systems we fix the meaning of the scores (Table I). The *oracle*, which reconstructs a correct workflow from the gold annotation, scores 1.00 on all three static layers and is safe on every executed swap on both splits, confirming that the tasks—including the held-out structural variants—are solvable and that the scorer rewards a genuinely correct solution. The *null* (empty output) and *random-node* baselines score 0.00 on every layer, so the metric cannot be gamed by degenerate or padded graphs. Real systems are therefore compared against a demonstrated ceiling of 1.00 and a floor of 0.00.

B. Baselines: structural validity does not imply safe executability

Across baselines the gap between structural validity and safe executability is large and consistent. On the held-out split **no baseline exceeds 0.33 safe-executability** (safety-instructed Gemini, 95% Wilson CI [0.24, 0.45]), and most sit near the floor. The layers fail for structurally different, quantified reasons (Section VII). The template-only baseline is graph-valid on the canonical prompts but has zero configuration completeness: well-formed graphs with no executable trade parameters. The deployed Koan generator is graph-valid on only 0.05 of held-out prompts, over-generating into a fixed suite that breaks required structure. The LLM baselines extract concrete configuration but omit safety machinery, discussed next.

TABLE I

STATIC STRUCTURAL/EXECUTABLE/SAFE METRICS VS. ON-CHAIN EXECUTION ON THE HELDOUT SPLIT ($n=75$ WORKFLOW PROMPTS). SAFE (CFG) IS REPORTED WITH A 95% WILSON INTERVAL. FORK-UNSAFE COUNTS WORKFLOWS THAT EXECUTED ON A LOCAL PY-EVM AT $>5\%$ OWN-TRADE PRICE IMPACT WITH NO PRICE-IMPACT GATE; FORK SAFE-RATE IS OVER PROMPTS WITH A DEFINITE ON-CHAIN OUTCOME. ORACLE AND THE NULL/RANDOM BASELINES BOUND THE CEILING AND FLOOR. LLM BASELINES RUN AT TEMPERATURE 0.

System	Graph valid	Exec. (cfg)	Safe (cfg) [95% CI]	Fork unsafe	Fork safe-rate
Oracle (ceiling)	1.00	1.00	1.00 [0.95,1.00]	0	1.00
Null (floor)	0.00	0.00	0.00 [0.00,0.05]	0	n/a
Random nodes	0.00	0.00	0.00 [0.00,0.05]	0	n/a
Template-only	0.52	0.00	0.00 [0.00,0.05]	0	n/a
Koan (rexex+gen)	0.05	0.00	0.00 [0.00,0.05]	0	n/a
Direct LLM (Gemini 3.1 FL)	0.09	0.05	0.03 [0.01,0.09]	19	0.67
Direct LLM (GPT-5.4 mini)	0.01	0.00	0.00 [0.00,0.05]	15	0.66
Constrained LLM (Gemini 3.1 FL)	0.31	0.25	0.13 [0.07,0.23]	18	0.68
Constrained LLM (GPT-5.4 mini)	0.15	0.12	0.01 [0.00,0.07]	18	0.63
Few-shot LLM (Gemini 3.1 FL)	0.17	0.15	0.12 [0.06,0.21]	19	0.66
Few-shot LLM (GPT-5.4 mini)	0.19	0.13	0.11 [0.06,0.20]	17	0.70
Safety-instruct LLM (Gemini 3.1 FL)	0.33	0.33	0.33 [0.24,0.45]	0	1.00
Safety-instruct LLM (GPT-5.4 mini)	0.07	0.07	0.07 [0.03,0.15]	0	1.00
Koan-Safe (rules)	0.52	0.43	0.43 [0.32,0.54]	0	1.00
Koan-Safe (LLM) (Gemini 3.1 FL)	0.71	0.64	0.64 [0.53,0.74]	0	1.00
Koan-Safe (LLM) (GPT-5.4 mini)	0.61	0.48	0.48 [0.37,0.59]	0	1.00
Koan-Safe (hybrid) (Gemini 3.1 FL)	0.73	0.67	0.67 [0.55,0.76]	0	1.00
Koan-Safe (hybrid) (GPT-5.4 mini)	0.64	0.57	0.57 [0.46,0.68]	0	1.00

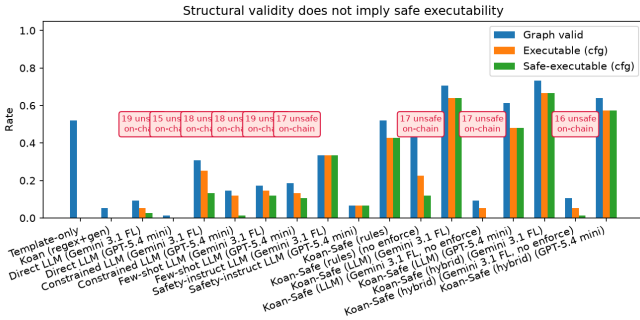


Fig. 1. Static structural, executable, and safe rates per system on the held-out test split ($n = 75$; reference floor/ceiling baselines omitted). Annotations mark workflows that executed unsafely on the local EVM.

C. On-chain evidence of unsafe execution

The static safe rate treats a declared-but-unparameterized safety node as a failure, but it cannot show that an “executable” workflow is actually dangerous. The execution layer does. Under direct, constrained, and few-shot prompting, both models emit swaps with a concrete amount but no wired price-impact gate; on the local EVM these **mine successfully at roughly 33% self-inflicted price impact**, and the harness labels them `unsafe_executed`—15 to 19 such executions per configuration on the held-out split (Table I). This is the thesis made concrete: a slippage bound, which the models often set, computes its minimum output from the already impact-adjusted quote and so does not prevent a large order from executing at severe self-inflicted impact; only a separate price-impact gate does. The template and Koan baselines never reach this failure because they never populate a concrete amount, so nothing executes.

D. Prompting alone does not close the gap

We ablate two mitigations on both models. Per-category structural *constraints* and a single *few-shot* example improve structural validity somewhat but leave 17–19 on-chain unsafe executions. Only an explicit *safety instruction* eliminates on-chain unsafe execution for both models (fork safe-rate 1.00) by inducing a parameterized price-impact gate—yet it still leaves most workflows structurally incomplete (safe-executability 0.33 for Gemini, 0.07 for GPT). No single prompt-level intervention closes both the structural and the safety half of the gap, which motivates enforcing safety outside the generator.

E. Koan-Safe closes the gap and generalizes

Table I shows that all three Koan-Safe generators lead the held-out split. The hybrid generator on Gemini reaches 0.67 safe-executability (95% CI [0.55, 0.76])—**double the best baseline** (0.33)—with the LLM generator at 0.64 and the rule-based generator at 0.43; crucially, *every* Koan-Safe configuration, on both model families, records **zero on-chain unsafe executions** (fork safe-rate 1.00) versus 15–19 for the direct, constrained, and few-shot LLMs.

The three generators also expose an honest limitation and its remedy. The frozen rule-based generator’s presets do not cover the held-out structural variants, so its structural validity falls from 1.00 on development to 0.52 on held-out. The language-model generator, unconstrained by fixed presets, generalizes to those variants (graph-valid 0.71), and the hybrid—LLM structure with parser-grounded configuration backfill—does best (0.73). The enforcement layer is what converts this structure into on-chain safety regardless of which generator produced it. On the development split, where the presets match, all three generators instead reach 1.00 structural validity and 0.55–0.58 safe-executability, again with zero unsafe executions.

TABLE II

ENFORCEMENT-LAYER ABLATION ON THE HELDOUT SPLIT: EACH KOAN-SAFE GENERATOR WITH THE SAFETY-ENFORCEMENT LAYER ON VS. OFF (SAME PARSER AND GENERATOR). THE LAYER IS WHAT DRIVES ON-CHAIN UNSAFE EXECUTIONS TO ZERO.

System	Safe (cfg)		Fork unsafe	
	off	on	off	on
Koan-Safe (rules)	0.12	0.43	17	0
Koan-Safe (LLM) (Gemini 3.1 FL)	0.00	0.64	17	0
Koan-Safe (hybrid) (Gemini 3.1 FL)	0.01	0.67	16	0

F. The enforcement layer is the cause

To attribute the effect we toggle only the enforcement layer, holding the parser and generator fixed (Table II). Disabling enforcement collapses safe-executability for every generator—on held-out, rule-based $0.43 \rightarrow 0.12$, LLM $0.64 \rightarrow 0.00$, hybrid $0.67 \rightarrow 0.01$ —and returns 16–17 on-chain unsafe executions, matching the plain LLM baselines. The same pattern holds on development ($13 \rightarrow 0$ unsafe for every generator). Since nothing but the enforcement layer changes between the on and off conditions, it, rather than the choice of generator, is what drives on-chain unsafe execution to zero.

G. Clarification and per-category behavior

On the clarification axis, Koan-Safe correctly handles 9 of 12 held-out underspecified or contradictory prompts; the three misses are terse requests (“I want a limit order”) that its frozen decision rule builds instead of questioning—an error we report rather than patch, since the method was frozen before the split existed. Per-category structural and safe rates (Table III) confirm the pattern: Koan-Safe’s advantage is uniform across categories on development, and on held-out its rule-based generator retains the canonical categories while the LLM and hybrid generators recover much of the structural-variant loss.

VII. ANALYSIS

A. Construct validity: does the static proxy measure real safety?

The static safe-executable layer is a cheap proxy for an expensive property: whether a workflow is actually safe to run. We test the proxy against the on-chain outcome on every prompt the fork harness executed or gated to a definite safe/unsafe label ($n = 891$ system $\times$ prompt pairs on the held-out split). The proxy is *sound* on this data: of the 282 workflows it declares safe-executable, all 282 are safe on-chain—*precision 1.00, zero false certifications*. Because Koan-Safe actually parameterizes a price-impact gate, these are genuine true positives, not an artifact of no system ever declaring safety (as was the case for the baselines alone). The proxy is also deliberately conservative: in 453 pairs it withholds “safe” from a workflow whose single executed swap happened to be within impact bounds, because it requires a declared gate rather than a fortunate trade size. The 156 proxy-unsafe/on-chain-unsafe pairs are the concrete unsafe executions it correctly refuses to certify. This is the intended direction of error for a safety metric: it never certifies danger as safe.

B. Failure taxonomy

Aggregating the per-prompt scores into named failure categories (released in full with the artifact) separates the systems’ error profiles on the held-out split. The deployed Koan generator’s failures are structural: 71 missing required edges and 54 missing required nodes, driven by fixed-suite over-generation. The LLM baselines’ failures are predominantly *safety-predicate* omissions: for direct Gemini the most frequent missing predicate is `price_impact_gate` (43 prompts), followed by `transaction_monitoring` (32) and `bridge_confirmation` (15). Koan-Safe’s enforcement layer targets exactly these: the hybrid generator’s remaining errors are almost entirely structural (`missing_required_edge` 20, driven by the held-out structural variants) with only 8 residual safety-predicate omissions in total, since the layer injects the safety policy the generator would otherwise omit.

C. Robustness to surface form and difficulty

The development split holds task semantics fixed across paraphrase clusters and tags each prompt with a difficulty tier, letting us separate competence from surface sensitivity. On development, within-cluster graph-valid agreement is high for Koan-Safe (rule-based 1.00) because enforcement makes structure deterministic, and lower for constrained Gemini (0.81), indicating that some of the LLM baseline’s structural score is surface-sensitive. Across difficulty tiers on the held-out split, the hybrid generator degrades gracefully (easy 1.00, medium 0.77, hard 0.25) and dominates the constrained-LLM baseline at every tier (easy 0.56, medium 0.26, hard 0.17); the hard tier is dominated by the novel structural variants, where the frozen rule-based generator falls furthest (hard 0.25) and the neural generators recover most of the loss.

VIII. DISCUSSION

The benchmark exposes a consistent division of labor among failure modes. Deterministic template instantiation guarantees structure but cannot bind trade-specific parameters, so its outputs are inert. The deployed Koan generator, driven by pattern templates, over-generates and mis-routes, which breaks structural validity even where its intent classification is correct. Language models invert this trade-off: they parse the request well enough to fill in concrete amounts, tokens, and prices, but they do not reliably reproduce the required graph structure, and, most importantly, they do not wire the safety machinery that would make an executable trade safe.

The ablations refine rather than overturn this picture. Structural constraints and few-shot examples improve graph validity but leave the on-chain unsafe executions intact; only an explicit safety instruction induces the missing price-impact gate and drives on-chain unsafe execution to zero. Yet even that intervention leaves most workflows structurally incomplete, so no single prompt-level fix closes both halves of the gap. This asymmetry—safety parameterization is promptable, but robust structure is not—is itself a finding about where the difficulty lies.

TABLE III
PER-CATEGORY GRAPH-VALID (G) AND SAFE-EXECUTABLE (S) RATE ON THE HELDOUT SPLIT. REFERENCE FLOOR/CEILING BASELINES (ALL CONSTANT) ARE OMITTED; SEE TABLE I.

System	Swap		Limit		Cross		Compo	
	G	S	G	S	G	S	G	S
Koan (regex+gen)	0.00	0.00	0.24	0.00	0.00	0.00	0.00	0.00
Direct LLM (Gemini 3.1 FL)	0.18	0.00	0.12	0.12	0.00	0.00	0.00	0.00
Direct LLM (GPT-5.4 mini)	0.00	0.00	0.00	0.00	0.07	0.00	0.00	0.00
Constrained LLM (Gemini 3.1 FL)	0.50	0.04	0.53	0.53	0.00	0.00	0.00	0.00
Constrained LLM (GPT-5.4 mini)	0.29	0.00	0.06	0.06	0.13	0.00	0.00	0.00
Few-shot LLM (Gemini 3.1 FL)	0.14	0.00	0.53	0.53	0.00	0.00	0.00	0.00
Few-shot LLM (GPT-5.4 mini)	0.21	0.07	0.35	0.35	0.13	0.00	0.00	0.00
Safety-instruct LLM (Gemini 3.1 FL)	0.61	0.61	0.41	0.41	0.00	0.00	0.07	0.07
Safety-instruct LLM (GPT-5.4 mini)	0.07	0.07	0.18	0.18	0.00	0.00	0.00	0.00
Koan-Safe (rules)	0.50	0.43	0.53	0.53	0.50	0.50	0.58	0.17
Koan-Safe (LLM) (Gemini 3.1 FL)	0.54	0.54	0.88	0.88	0.72	0.72	0.83	0.42
Koan-Safe (LLM) (GPT-5.4 mini)	0.54	0.54	0.76	0.71	0.61	0.44	0.58	0.08
Koan-Safe (hybrid) (Gemini 3.1 FL)	0.54	0.54	1.00	1.00	0.72	0.72	0.83	0.42
Koan-Safe (hybrid) (GPT-5.4 mini)	0.50	0.50	0.88	0.88	0.67	0.67	0.58	0.17

The on-chain result sharpens the practical stakes. A workflow that a graph-only or even a configuration-level evaluator would accept as “executable” can, when actually run, mine a trade at severe self-inflicted price impact. This happens precisely because the systems conflate two different protections: a slippage bound, which guards against price movement between quote and execution, and a price-impact gate, which guards against the cost of one’s own order. Templates encode a static threshold but have no live trade to protect; language models supply the live trade but omit the gate. Neither composition is safe.

These observations suggest that safe DeFi workflow synthesis should be evaluated, and built, as a constraint-satisfaction problem over executable state rather than as graph generation. A generator that emits a structurally plausible, parameter-complete workflow is not done: the workflow must additionally carry, and enforce, the safety predicates that the target protocols require. Our benchmark operationalizes exactly this requirement and makes the gap measurable.

Koan-Safe is a direct consequence of this framing, and its results support it. Treating safety as an enforcement pass over a candidate workflow—rather than as an emergent property of the generator—doubles safe-executability over the best baseline on a held-out split and eliminates on-chain unsafe execution across three different generators and two model families. The enforcement on/off ablation shows the layer, not the generator, is responsible. Two qualifications keep the claim honest. First, the layer improves safety but not intent: it never invents a missing amount or price, so a request that omits them still yields a non-executable workflow, which is the correct conservative behavior but caps the achievable rate. Second, the split between generators is informative: the frozen rule-based presets do not cover novel workflow structures and lose structural validity on the held-out variants, whereas a language-model generator under the same enforcement layer recovers most of that loss. The practical recommendation is therefore a hybrid: use a flexible generator for structure and a deterministic enforcement layer for safety, and evaluate the

composition on executable ground truth rather than graph shape.

IX. LIMITATIONS

a) Local EVM, not a mainnet fork.: The execution layer deploys a constant-product AMM and mock tokens on an in-process EVM with deterministically seeded reserves. This yields real EVM execution semantics (real transfers, reverts, and gas) and byte-for-byte reproducibility, but the prices and liquidity are synthetic. It is a pinned local snapshot rather than a live mainnet fork; absolute price-impact magnitudes depend on the seeded reserves, though the qualitative unsafe-execution finding does not. Replacing the synthetic AMM with a mainnet-fork RPC is a natural extension and the harness interface is designed for that substitution.

b) Scale, single annotator, and scoring strictness.: The development split uses 120 expert-authored prompts and the held-out split 87, with temperature-0 decoding and a single run per model on two model families. Temperature-0 API calls still vary slightly across runs, so individual decimals are approximate; we report every rate with a confidence interval and generate all tables from saved outputs. The gold annotations were authored primarily by one team; we provide a second-annotator protocol, a reproducible stratified subset, and tooling to compute inter-annotator agreement, but the agreement number awaits an independent pass and is therefore not yet reported. Edge matching is strict and type-level, which penalizes reasonable-but-reordered graphs and treats structural rates as a lower bound; this does not affect the safe-rate or the on-chain unsafe-execution findings, which are driven by missing safety parameterization rather than edge ordering.

c) Scope of the Koan-Safe method.: Koan-Safe encodes engineered domain knowledge—a token dictionary, per-category node presets, and conservative safety defaults—and reads only the raw prompt, but it has no access to gold annotations. Its enforcement layer improves safety, not intent extraction: it never fabricates a missing amount, price, or chain, so its achievable executable rate is bounded by what the

parser and generator can read from the prompt. Its rule-based generator’s fixed presets do not cover workflow structures outside the four categories, which is visible as reduced structural validity on the held-out variants; the language-model and hybrid generators mitigate but do not fully remove this. The held-out split is authored by the same team as the method, so it tests structural and surface generalization but not adversarial distribution shift from an independent party.

d) Category scope of the execution layer: On-chain execution currently covers swaps and the swap leg of compositional workflows; limit orders are checked for immediate fillability against the pool mid-price, and cross-chain workflows are not executed because a single local chain cannot bridge. Every unexecuted case is recorded explicitly rather than silently passed.

X. ETHICS AND RESPONSIBLE DISCLOSURE

This work is an offline evaluation. All experiments run against a local, in-process EVM with mock tokens and seeded liquidity; no transaction in this paper touches a live network or real funds. The benchmark is intended to help identify unsafe generated workflows before execution, not to encourage users to execute generated DeFi workflows with real assets. Any future extension to a mainnet-fork or live setting must keep simulation strictly separated from real execution, use test settings, and disclose residual risks. The evaluated no-code system and the language models are used as-is through their public interfaces; we report their failures to motivate safer synthesis, not to single out any implementation. Model outputs and derived metrics are released so that the results can be independently reproduced and audited.

REFERENCES

- [1] S. Qiao *et al.*, “Benchmarking agentic workflow generation,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.07869.
- [2] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world apis,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.16789.
- [3] M. Li *et al.*, “API-Bank: A comprehensive benchmark for tool-augmented LLMs,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2304.08244.
- [4] H. Adams, N. Zinsmeister, and D. Robinson, “Uniswap v2 core,” Uniswap, Tech. Rep., 2020, whitepaper.